

Analiza descendenta. Gramatici LL(k) - eliminare
recursivitate stanga. factorizare.

Table of Contents

Eliminare recursivitate stanga

Productii ϵ

Factorizare stanga

Algoritm de parsare bazat pe tabel

Teorema

4.2.2, 4.2.3 Teorema. O gramatica LL(k) nu poate avea simbol nonterminal recursiv stanga.

Daca $X \Rightarrow X\omega, \omega \neq \varepsilon$ - X nonterminal recursiv stanga

► $E \rightarrow E + T \mid T$

► $T \rightarrow T * F \mid F$

► $F \rightarrow (E) \mid id$

are doua productii cu recursivitate stanga

Teorema

Teorema. Pentru orice gramatica CFG $G = (T, N, P, Z)$ cu simboluri nonterminale recursive stanga exista o gramatica echivalenta $G' = (T, N', P', Z)$ fara nonterminale recursive stanga.

Idee

$$X \rightarrow X\alpha|\beta \text{ devine } \begin{cases} X \rightarrow \beta X' \\ X' \rightarrow \alpha X'|\varepsilon \end{cases}$$

$$\blacktriangleright E \rightarrow E + T|T$$

$$\blacktriangleright T \rightarrow T * F|F$$

$$\blacktriangleright F \rightarrow (E)|id$$

$$\blacktriangleright E \rightarrow TE'$$

$$\blacktriangleright E' \rightarrow +TE'|\varepsilon$$

$$\blacktriangleright T \rightarrow FT'$$

$$\blacktriangleright T' \rightarrow *FT'|\varepsilon$$

$$\blacktriangleright F \rightarrow (E)|id$$

Dar...

NU intra la examen
vezi Testare online

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$

$$S \Rightarrow Aa \Rightarrow Sda$$

ne trebuie un algoritm care sa elimine toate nonterminalele cu
recursivitate stanga

- ▶ Consideram ca $N = \{X_1, X_2, \dots, X_n\}$ - simbolurile nonterminale sunt numerotate consecutiv.
- ▶ Daca putem alege indicii a.i. indicii sa respecte $i < j$ pentru toate productiile $X_i \rightarrow X_j \omega$ atunci G nu are recursivitate stanga.
- ▶ Daca o astfel de numerotare nu este posibila pentru G , atunci se genereaza G' .

Exemple:

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$
- ▶ Daca S e 1, A e 2, prima productie respecta $i < j$ dar nu si a doua
- ▶ $E \rightarrow E + T$ nu respecta $i < j$

Algoritm de eliminare recursivitate stanga

NU intra la examen Testare online

1. Fie $N' = N, P' = P$. Se executa pasii 2,3 pentru $i = 1, \dots, n$
2. Pentru $j = 1, \dots, i - 1$
 $X_i \rightarrow X_j \omega \in P'$ se inlocuiesc cu $\{X_i \rightarrow \chi_j \omega \mid X_j \rightarrow \chi_j \in P'\}$.
In consecinta, $X_i \Rightarrow^+ X_j \gamma$ implica $i \leq j$.
3. Se inlocuiesc $X_i \rightarrow X_i \omega \in P'$ cu $\{Y_i \rightarrow \omega Y_i\} \cup \{Y_i \rightarrow \varepsilon\}$
adaugand un nou simbol Y_i la N' .
+ se inlocuiesc $X_i \rightarrow \chi, \chi \neq X_i \gamma$ cu $X_i \rightarrow \chi Y_i$.
Simbolurile noi se numereaza cu $n+1, n+2, \dots$

► $E \rightarrow E + T | T$

► $T \rightarrow T * F | F$

► $F \rightarrow (E) | id$

presupunem ordinea $E(1) < T(2) < F(3)$

i	pasul 2	pasul 3	variabila noua
1	nu se executa	$E \rightarrow E + T T$ devin $E' \rightarrow +TE' \varepsilon$ si $E \rightarrow TE'$;	$E'(4)$
2	$j = 1$ $T \rightarrow E\omega$ nu exista	$T \rightarrow T * F F$ devin $T' \rightarrow *FT' \varepsilon$ $T \rightarrow FT'$	$T'(5)$
3	$j = 1, 2$ $F \rightarrow E\omega$ sau $F \rightarrow T\omega$ nu exista	$F \rightarrow F\omega$ nu exista	
4,5	nu se modifica nimic		

Rezultat:

- ▶ $E \rightarrow E + T \mid T$
 - ▶ $T \rightarrow T * F \mid F$
 - ▶ $F \rightarrow (E) \mid id$
- ▶ $E \rightarrow TE'$
 - ▶ $E' \rightarrow +TE' \mid \varepsilon$
 - ▶ $T \rightarrow FT'$
 - ▶ $T' \rightarrow *FT' \mid \varepsilon$
 - ▶ $F \rightarrow (E) \mid id$

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$

...pasul 2 al algoritmului: Pentru $j = 1, \dots, i - 1$ $X_i \rightarrow X_j\omega \in P'$ se inlocuiesc cu $\{X_i \rightarrow \chi_j\omega | X_j \rightarrow \chi_j \in P'\}$.

Daca $S(1) < A(2)$

- ▶ $i = 1$ nimic
- ▶ $i = 2$ la pasul 2 $A \rightarrow Sd$ se inlocuieste cu $\{A \rightarrow Aad|bd\}$

- ▶ $S \rightarrow Aa|b$
- ▶ $A \rightarrow Ac|Sd|\varepsilon$

...pasul 3 al algoritmului: Se inlocuiesc $X_i \rightarrow X_i\omega \in P'$ cu $\{Y_i \rightarrow \omega Y_i\} \cup \{Y_i \rightarrow \varepsilon\}$ adaugand un nou simbol Y_i la N' .
 + se inlocuiesc $X_i \rightarrow \chi, \chi \neq X_i\gamma$ cu $X_i \rightarrow \chi Y_i$.

Daca $S(1) < A(2)$

- ▶ $i = 1$ nimic
- ▶ $i = 2$ la pasul 2 $A \rightarrow Sd$ se inlocuieste cu $\{A \rightarrow Aad|bd\}$
- ▶ $i = 2$ la pasul 3 $A \rightarrow Ac|Aad|bd|\varepsilon$ se inlocuieste cu $A' \rightarrow cA', A' \rightarrow adA', A' \rightarrow \varepsilon$ si $A \rightarrow bdA', A \rightarrow A'$

Teorema. Daca sirul ω din $X_i \rightarrow X_i\omega$ nu incepe cu $X_j, j \leq i$ atunci $X_i \rightarrow X_i\omega$ se poate inlocui cu $\{Y_i \rightarrow \omega, Y_i \rightarrow \omega Y_i\}$ si $X_i \rightarrow \chi$ cu $\{X_i \rightarrow \chi, X_i \rightarrow \chi Y_i\}$ la pasul 3.

pasul 3 anterior ...se inlocuiesc $X_i \rightarrow X_i\omega \in P'$ cu $\{Y_i \rightarrow \omega Y_i\} \cup \{Y_i \rightarrow \varepsilon\}$ adaugand un nou simbol Y_i la N' .
+ se inlocuiesc $X_i \rightarrow \chi, \chi \neq X_i\gamma$ cu $X_i \rightarrow \chi Y_i$.

Se evita introducerea productiilor ε .

Table of Contents

Eliminare recursivitate stanga

Productii ε

Factorizare stanga

Algoritm de parsare bazat pe tabel

Intra la examen

- ▶ $E \rightarrow E + T \mid T$
- ▶ $T \rightarrow T * F \mid F$
- ▶ $F \rightarrow (E) \mid id$

Cu productii ε

- ▶ $E \rightarrow TE'$
- ▶ $E' \rightarrow +TE' \mid \varepsilon$
- ▶ $T \rightarrow FT'$
- ▶ $T' \rightarrow *FT' \mid \varepsilon$
- ▶ $F \rightarrow (E) \mid id$

Fara productii ε

- ▶ $E \rightarrow TE' \mid T$
- ▶ $E' \rightarrow +T \mid +TE'$
- ▶ $T \rightarrow FT' \mid F$
- ▶ $T' \rightarrow *F \mid *FT'$
- ▶ $F \rightarrow (E) \mid id$

Observatii

- ▶ Recursivitatea stanga precum $E \rightarrow T | E + T$ - utilizata pentru a reflecta asociativitatea stanga a operatorilor.
- ▶ Aceeasi proprietate avem si in $E \rightarrow TE', E' \rightarrow +TE', E' \rightarrow \varepsilon$
- ▶ Insa asociativitate dreapta $E \rightarrow T, E \rightarrow T + E$.

Productii ε

Productiile ε se pot elimina intotdeauna dintr-o gramatica LL(k),
dar aceasta poate mari valoarea lui k.

4.2.3

Teorema

TEOREMA. Pentru orice gramatica LL(k) cu productii ε exista o gramatica LL($k+1$) fara productii ε care genereaza limbajul $L(G) - \varepsilon$.

Prin introducerea productiilor ε se poate reduce k .

Teorema

TEOREMA. Pentru orice gramatica $LL(k+1)$, $k > 0$ fara productii ε exista o gramatica $LL(k)$ echivalenta cu productii ε .

Table of Contents

Eliminare recursivitate stanga

Productii ϵ

Factorizare stanga

Algoritm de parsare bazat pe tabel

Factorizare stanga

$$\text{Fie } P = \left\{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow Yc|Yd \\ Y \rightarrow a|bY \end{array} \right\}$$

Factorizare stanga

$$\text{Fie } P = \{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow Yc | Yd \\ Y \rightarrow a | bY \end{array} \}$$

Productiile $X \rightarrow Yc$ si $X \rightarrow Yd$ nu pot fi distinse chiar prin examinarea oricarui numar fix de simboluri din sirul de intrare deoarece din Y se poate deriva un sir de lungime si mai mare.

Solutie: evitarea problemei prin amanarea deciziei. Ambele incep cu Y , nu trebuie facuta distinctie intre ele decat dupa ce Y a fost recunoscut.

Factorizare stanga

$$\text{Fie } P = \{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow Yc|Yd \\ Y \rightarrow a|bY \end{array} \}$$

devine

$$\text{Fie } P = \{ \begin{array}{l} Z \rightarrow X \\ X \rightarrow YX' \\ X' \rightarrow c|d \\ Y \rightarrow a|bY \end{array} \}$$

Se poate examina un singur caracter inainte pt a face diferente
intre cele doua variante c sau d .

Factorizare stanga

Fie $P = \{$

- $Z \rightarrow X$
- $X \rightarrow \text{if } E \text{ then } S | \text{if } E \text{ then } S \text{ else } S | a$
- $E \rightarrow b\}$

devine

Fie $P = \{$

- $Z \rightarrow X$
- $X \rightarrow \text{if } E \text{ then } S \ S' | a$
- $S' \rightarrow \text{else } S | \varepsilon$
- $E \rightarrow b\}$

Table of Contents

Eliminare recursivitate stanga

Productii ϵ

Factorizare stanga

Algoritm de parsare bazat pe tabel

Parsing table - tabel de derivare

- ▶ Ullman 4.4 . Nonrecursive predictive parsing
- ▶ Table-driven predictive parsing: input, stiva, parsing table.
- ▶ Tabel de derivare: $M[A,a]$ - A nonterminal, a - terminal sau $\#$

Exemplu de tabel de derivare

	lookahead					
	i	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

$$\begin{aligned}
 P = \{ & E \rightarrow TE' \\
 & E' \rightarrow +TE' | \varepsilon \\
 & T \rightarrow FT' \\
 & T' \rightarrow *FT' | \varepsilon \\
 & F \rightarrow (E) | i \}
 \end{aligned}$$

Algoritm de derivare predictiva cu tabel de derivare

```
#S (simbol de start) pe stiva, string# la intrare
set ip (input pointer) to point to the first symbol of
input string
repeat
  let X be the top stack symbol and a the symbol pointed to
  by ip
  if X is terminal or # then
    if X = a then
      pop X from the stack and advance ip
    else error()
  else
    if M[X,a] = X-> Y1 Y2 ...Yk then begin
      pop X from the stack
      push Yk, Yk-1, ...Y1 onto the stack, with Y1 on top
      output the production X-> Y1 Y2 ...Yk
    end
    else error()
until X=#
```

Algoritm de derivare predictiva cu tabel de derivare

```
#S (simbol de start) pe stiva, string# la intrare
set ip (input pointer) to point to the first symbol of
input string
repeat
  let X be the top stack symbol and a the symbol pointed to
  by ip
  if X is terminal or # then
    if X = a then
      pop X from the stack and advance ip
    else error()
  else
    if M[X,a] = X-> Y1 Y2 ...Yk then begin
      pop X from the stack
      push Yk, Yk-1, ...Y1 onto the stack, with Y1 on top
      output the production X-> Y1 Y2 ...Yk
    end
    else error()
until X=#
```

$$\{tgt \rightarrow q | t \in T\} \cup$$

$$\{Xq \rightarrow x_n \dots x_1 q | X \rightarrow x_1 x_2 \dots x_n \in P, n \geq 0, X \in N, X_i \in V\}$$

Algoritm de constructie a tabelului

	lookahead					
	id	+	*	(	)	#
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow i$			$F \rightarrow (E)$		

$$\begin{aligned}
 P = \{ & E \rightarrow TE' \\
 & E' \rightarrow +TE' | \varepsilon \\
 & T \rightarrow FT' \\
 & T' \rightarrow *FT' | \varepsilon \\
 & F \rightarrow (E) | i \}
 \end{aligned}$$

simbol	$FIRST_1(X)$	$FOLLOW_1(X)$
E	$\{ (, i \}$	$\{), \# \}$
E'	$\{ +, \varepsilon \}$	$\{), \# \}$
T	$\{ (, i \}$	$\{ +, \#,) \}$
T'	$\{ *, \varepsilon \}$	$\{ +, \#,) \}$
F	$\{ (, i \}$	$\{ *, +, \#,) \}$

- for each production $A \rightarrow \alpha$ do steps 2 and 3
- for each terminal a in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
- if $\varepsilon \in FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, b]$ for each terminal $b \in FOLLOW(A)$.
if $\varepsilon \in FIRST(\alpha)$ and $\# \in FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \#]$
- Make each undefined entry of M be error